Verteilte Systeme
Portfolioaufgaben 2026

Prüfungshinweise

Bearbeitung in Eigenleistung

Alle Aufgaben sind in Eigenleistung zu bearbeiten. Die Lösungen müssen einen individuellen Lösungsweg erkennen lassen. Fremde Quellen dürfen zwar als Hilfestellung genutzt werden. Die Lösungen müssen aber dennoch selbst ausgearbeitet werden. Abschreiben (auch mit Umformulierungen) gilt als Täuschungsversuch.

Quellen und KI-Sprachassistenten

Fremde Quellen dürfen verwendet werden, müssen aber durch Kommentare im Quellcode mit einem kurzen Verweis auf die Quelle und die verwendeten Inhalte gekennzeichnet werden.

Eine besondere Regel gilt für KI-Sprachassistenten: Richtig eingesetzt können sie sehr nützlich sein, weshalb die Verwendung natürlich erlaubt ist, aber als Quellenangabe gekennzeichnet werden muss. KI-Chatverläufe müssen vollständig gesichert und dem Portfolio beigelegt werden (als PDF-Export oder als öffentlicher Link auf den Chat). Bei begründetem Verdacht auf unzureichend gekennzeichnete KI-Nutzung gibt es Punktabzug oder die Prüfung wird nicht bestanden!

Hier ein paar Beispiele, was erlaubt ist:

- **Klärung konkreter Fragen:** „*Wie lässt sich das HATEOAS-Prinzip für REST-Webservices umsetzen?*“
- **Recherche eines Themas:** „*Welche Möglichkeiten gibt es, in Node.js auf Datenbanken zuzugreifen?*“
- **Prüfen von Annahmen:** „*Stimmt es, dass Webservice-Aufrufe immer synchron erfolgen?*“
- **Hilfestellung beim Programmieren:** „*Warum funktioniert folgender Quellcode nicht?*“
- **Generieren von Kommentaren:** „*Schreibe einen JSDoc-Kommentar zu dieser Funktion.*“
- **Hilfe bei der Fehlersuche:** „*Was bedeutet der Fehler ‚Cannot read properties of undefined‘?*“

Quellcode muss grundsätzlich selbst geschrieben werden. Ebenso sind Sie angehalten, Chat-Assistenten als Werkzeug zu nutzen und die Antworten stets zu hinterfragen. Stellen Sie deshalb möglichst viele Folgefragen und weißen Sie darauf hin, wenn etwas unklar ist oder eine Antwort nicht stimmig erscheint. Bedenken Sie, dass Chat-Assistenten in Wirklichkeit „statistische Textgeneratoren“ sind und nicht wirklich „denken“ oder „etwas verstehen“ können. Nicht erlaubt sind deshalb:

- Quellcode, der über mehr als kleine Hilfsfunktionen hinaus geht, von der KI generieren lassen.
- Große Teile der Aufgabenstellung ohne eigene Anstrengung von der KI lösen zu lassen.
- Vibe-Coding, wenn es nicht explizit bei einer Teilaufgabe zugelassen wird.
- KI-generierte Inhalte unreflektiert und ungeprüft zu übernehmen.

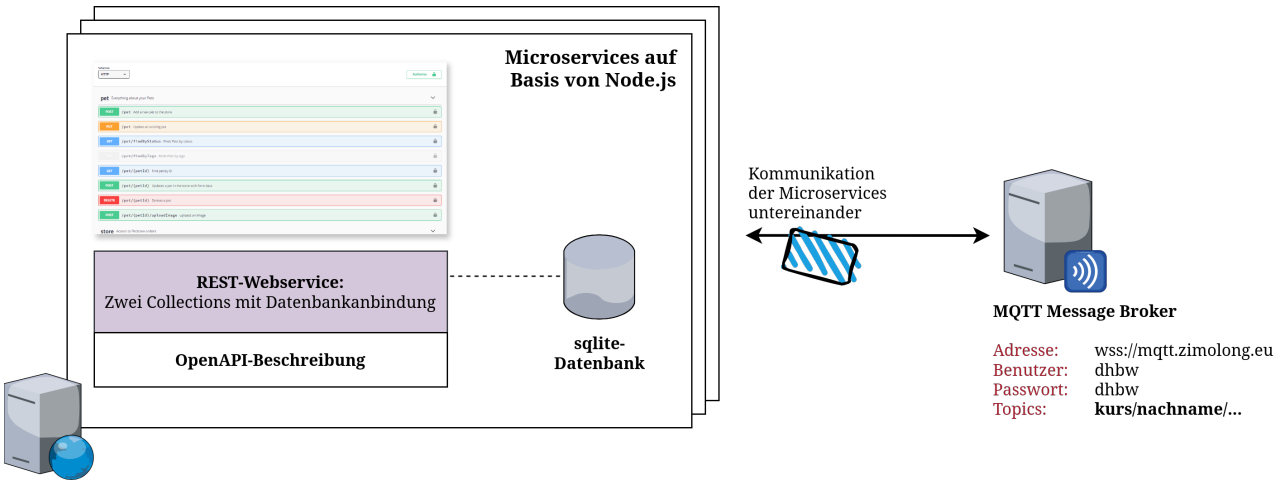
Denn am Ende geht es nicht einfach darum, die richtige Antwort zu liefern, sondern anhand der Aufgabenstellung etwas zu lernen. Die Nichteinhaltung führt zu Nichtbestehen der Prüfung!

Abgabe in Moodle

Die Abgabe erfolgt per Upload in Moodle. Quellcodes müssen, auch wenn Sie online verfügbar sind, ebenfalls in Moodle hochgeladen werden. Denken Sie daran, das Verzeichnis `node_modules` vor dem Erstellen der ZIP-Datei zu löschen! Ein Git-Repository wird nicht benötigt, kann aber angelegt werden, wenn Sie wollen. Textdokumente und Office-Dateien müssen im PDF-Format abgegeben werden. Gehen Sie bitte nicht davon aus, dass die Lehrenden dieselben Softwarepakete wie Sie installiert haben.

Architektur der zu erstellenden Anwendung

In diesem Projekt soll das theoretische Wissen aus den Vorlesungen praktisch angewendet werden. Ihr Auftrag ist daher, eine verteilte Anwendung mit einer Microservice-Architektur zu implementieren. Allerdings konzentrieren wir uns dabei nur auf das Backend (also die Serverseite), da die Frontendprogrammierung bereits letztes Semester behandelt wurde. In einem echten Projekt müssten Sie natürlich auch eine Webanwendung oder eine Mobile App als Benutzeroberfläche programmieren. Die folgende Abbildung zeigt die Architektur der Anwendung:



Die Anwendung soll aus mindestens zwei Microservices bestehen, die jeweils einen klar abgegrenzten Funktionsbereich abdecken. Die Microservices sollen als 12-Faktor-Apps ausgeführt sein und daher jeweils ein eigenes Quellcodeverzeichnis besitzen und als eigenständige Programme gestartet werden. Zusätzlich gilt:

- Jeder Microservice besitzt eine eigene Datenbank, in der seine Daten liegen.
- Jede Datenbankentität gehört exakt einem Microservice und darf von den anderen nicht dupliziert werden.
- Jeder Microservice besitzt eine eigene Schnittstelle für entfernte Zugriff auf seine Funktionen/Daten.

Man spricht hier auch davon, eine größere Anwendung nicht monolithisch auszuführen (alle Funktionen als Teil eines großen Programms), sondern in Microservices zu zerlegen. Jeder Microservice deckt dann einen jeweils anderen Funktionsbereich ab und kommuniziert hierfür über das Netz mit den anderen Microservices. Erst die Summe aller Microservices ergibt die Anwendung. Folgende Leitfragen helfen beim Zuschnitt der Microservices:

- *Welche fachlich abgrenzbaren Teilbereiche besitzt die Anwendung?*

  Zum Beispiel in einem Webshop: Produktkatalog, Bestellabwicklung, Lagerhaltung, Versand/Retouren-Abwicklung, … Man spricht hier auch von der Domänengrenze (Domain Boundary), welche die Trennung der Microservices bestimmt. In einem Unternehmen handelt es sich meistens um getrennte Abteilungen. Und auch die Microservices werden typischerweise von getrennten Teams entwickelt.

- *Welche Teile der Anwendung werden wie stark beansprucht?*

  Nicht alle Teile einer Anwendung werden gleich häufig genutzt. In einem Webshop werden zum Beispiel viel mehr Artikel angeschaut, als tatsächlich gekauft werden. Trennt man diese beiden Teile daher in zwei Microservices, können beide unabhängig voneinander deployed und skaliert werden.

- *Gibt es besondere technische Anforderungen?*

  Manche Teile einer Anwendung setzen eine bestimmte Laufzeitumgebung oder Bibliotheken voraus, die es nur für eine bestimmte Programmiersprache gibt. Machine-Learning-Anwendungen werden zum Beispiel häufig in Python geschrieben, weshalb es für diese Sprache besonders viele Bibliotheken dazu gibt. Microservices bieten die hierfür notwendige Flexibilität, um für jeden Teil der Anwendung die am besten passende Technologie (sofern man sie beherrscht) einsetzen zu können.

Teilaufgaben

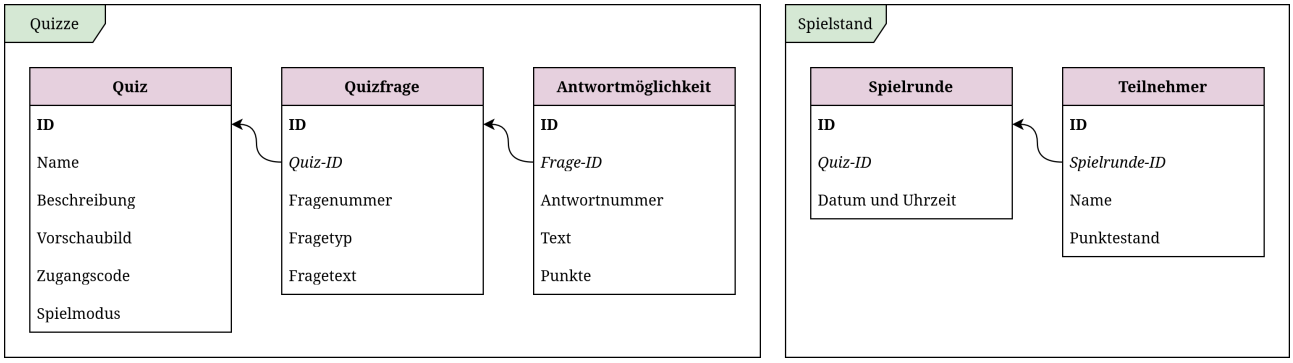
Bitte beachten Sie, dass der Portfolioanteil im vierten Semester eine **Einzelleistung** ist, die mit der Gruppenarbeit des letzten Semesters verrechnet wird. Da die Portfolionote überwiegend personenbezogen gebildet werden muss, muss dieses Semester jede Person eine eigene Lösung erarbeiten und abgeben.

Aufgabe 1: Datenmodell (15 Punkte)

Definieren Sie zunächst den Anwendungsfall und beschreiben Sie in ein paar Sätzen, welche Problemstellung Ihre Anwendung lösen soll und welche Funktionen sie hierfür besitzen muss. Die Funktionen können Sie als Fließtext oder in Form von User Stories beschreiben. Es genügt, wenn Sie sich dabei auf funktionale Aspekte ohne Bezug zu einer grafischen Benutzeroberfläche beziehen. Zum Beispiel:

- Als Lehrperson will ich Quizze anlegen können, um meinen Studierenden die Möglichkeit zu geben, ihr Wissen zu überprüfen.
- Als Lehrperson will ich verschiedene Arten von Quizfragen wie Single Choice, Multiple Choice, … anlegen können, um die Quizze abwechslungsreich zu gestalten.
- …

Dokumentieren Sie mindestens fünf Anforderungen auf diese Weise. Anschließend entwerfen ein dazu passendes Datenmodell und teilen die Entitäten entlang funktionaler Grenzen in mehrere Microservices auf. Dokumentieren Sie das Datenmodell als UML-Diagramm wie im folgenden Beispiel.



Aufgabe 2: REST-Schnittstelle (25 Punkte)

Entwerfen Sie nun eine zum Datenmodell passende REST-Schnittstelle und dokumentieren diese im OpenAPI-Format. Für ein Beispiel gehen Sie auf https://editor.swagger.io/ und wählen den Menüpunkt *„File → Load Example → OpenAPI 3.1 Petstore“* aus. Sie müssen nicht alle Entitäten über die REST-Schnittstelle verfügbar machen. Oft ist das auch gar nicht sinnvoll. Aber folgendes müssen Sie sich überlegen:

- Welche Daten müssen das UI oder die anderen Microservices lesen können?
- Welche Daten muss das UI direkt ändern können (z.B. um eine neue Spielrunde zu starten)?
- Wie werden die Daten auf URLs abgebildet?
- Welche HTTP-Methoden müssen je URL verarbeitet werden können?
- Wie sehen die Nutzdaten aus, die beim Aufruf einer URL ausgetauscht werden?
- Welche HTTP-Statuscodes können bei jedem Aufruf auftreten?

Beim obigen Beispiel macht es zum Beispiel keinen Sinn, dass der Spielstand-Microservice beide Entitäten verfügbar macht. Besser wäre, wenn man nur eine neue Spielrunde, die dann die Ergebnisse alle Teilnehmer mit beinhaltet, anlegen und auch wieder abrufen könnte.

Aufgabe 3: Implementierung (50 Punkte)

SOLID-Kriterien: Die Microservices sollen in Node.js, so wie in der Vorlesung gezeigt, programmiert werden. Dabei ist auf eine saubere Trennung der Zuständigkeiten nach den SOLID-Prinzipien zu achten. Dies wird in der Vorlesung behandelt und ist im Skript ausführlich beschrieben (Stichwort „Strukturierung des Quellcodes“). Konkret bedeutet dies, dass der Quellcode in allgemeine Funktionen, HTTP-Handler/Controller und fachliche Serviceklassen/Dateien unterteilt werden soll.

sqlite-Datenbank: Als Datenbank soll eine eingebettete sqlite-Datenbank verwendet werden. Hierfür muss dem Node.js-Projekt das Paket sqlite als Abhängigkeit hinzugefügt werden. Bei ändernden Datenbankzugriffen soll die referentielle Integrität beachtet werden, z.B. mit **`ON DELETE CASCADE`**.

Hinweis: sqlite ignoriert standardmäßig referentielle Abhängigkeiten zwischen den Tabellen. Um dies zu ändern, muss beim Programmstart der Befehl **`PRAGMA foreign_keys = ON`** an die Datenbank geschickt werden.

REST-Schnittstelle: Jeder Microservice soll eine REST-Schnittstelle für öffentliche Zugriffe auf seine Funktionen besitzen. Die in der Vorlesung behandelten REST-Prinzipien sind vollständig einzuhalten.

OpenAPI-Beschreibung: Die REST-Schnittstelle jedes Microservices soll als OpenAPI-Spezifikation beschrieben werden. Diese können Sie entweder mit der Webanwendung Apicurito grafisch erstellen (empfohlen, um OpenAPI wirklich zu verstehen), händisch schreiben oder mit KI generieren. Die Beschreibung muss die REST-Schnittstelle vollständig und exakt beschreiben. Die Dokumentation muss zum Code passen. ☺

Gegenseitige Lesezugriffe: Die Microservices dürfen die REST-Endpunkte der anderen Microservices aufrufen, um fremde Daten zu lesen. Hierfür sollte jeder Microservice über das Paket dotenv eine `.env`-Datei mit Umgebungsvariablen einlesen, welche die Zieladressen der benötigten anderen Microservices enthält. Jedoch darf die Bearbeitung einer Anfrage nicht gänzlich fehlschlagen, wenn ein entfernter Microservice nicht erreichbar ist. Entfernte Aufrufe müssen daher grundsätzlich fehlertolerant sein.

Event-basierte Kommunikation: Nicht erlaubt ist, dass ein Microservice die REST-Schnittstelle eines anderen Microservice aufruft, um Änderungen auf der anderen Seite zu bewirken (beispielsweise Daten im anderen Microservice zu löschen oder zu aktualisieren), da dies die Kopplung zwischen den Microservices erhöht. Stattdessen soll jeder Microservice bei einer Änderung seiner eigenen Daten ein Event an einen MQTT-Broker senden, das die Art der Änderung und die notwendigen Schlüsselwerte beinhaltet. Zum Beispiel:

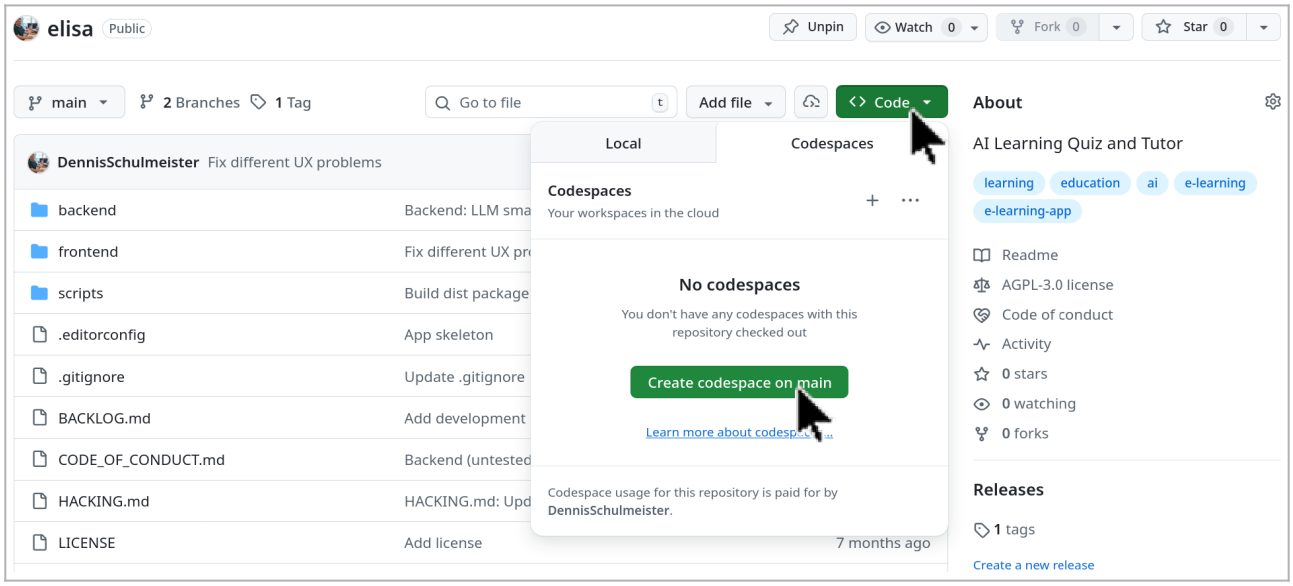
```
{"event": "update",  "entity": "GameRound",  "id": "4711-0815"}
```

Die anderen Microservices sollen die für sie notwendigen Events abfangen und ggf. notwendige Änderungen auf ihrer Seite eigenständig durchführen. Dadurch ist sichergestellt, dass jeder Microservice maximal unabhängig von den anderen funktionieren kann.

Aufgabe 4: Docker Compose (10 Punkte)

Fügen Sie dem Projekt zum Schluss noch eine **`docker-compose.yml`**-Datei hinzu, die es ermöglicht, alle Microservices sowie alle eventuell zusätzlich benötigten Softwarepakete zusammen zu deployen und auszuführen. Hierfür sollte jeder Microservice ein **`Dockerfile`** besitzen, um ihn als Container zu verpacken. Mit Docker Compose sollen die Container dann gemeinsam gestartet werden können. Ein Beispiel hierfür finden Sie im Skript unter der Überschrift „Mehrere Container orchestrieren mit Docker Compose“. Auf GitHub haben wir ein weiteres Beispiel für Sie vorbereitet: Docker-Beispiel

Da die Installation von Docker unter Windows leider etwas umständlich ist (wenn Sie nicht WSL verwenden), können Sie diesen Teil der Aufgabe online mit GitHub Codespaces bearbeiten. Hierfür müssen Sie für das gesamte Projekt ein Git-Repository anlegen und auf GitHub hochladen (am besten als privates Repository). Anschließend betrachten Sie das Repository auf GitHub und klicken auf *„Code → Codespaces → Create codespace on …“*, um es online zu bearbeiten.



Sie erhalten dadurch Zugriff auf VS Code Online innerhalb einer Linux-VM, die bereits alle Werkzeuge vorinstalliert hat. Dort wird das Repository automatisch in ein neues Verzeichnis geklont, so dass sie es direkt bearbeiten können. Beachten Sie dabei aber, dass alle Änderungen commited und zurück auf GitHub gepusht werden müssen, um nicht verloren zu gehen!

Bewertungskriterien

Die nachfolgende Checkliste soll Ihnen helfen, Ihre Lösung zu überprüfen. Dieselben Kriterien werden später auch bei der Bewertung der Aufgaben verwendet.

Eigenleistung und KI-Nutzung

- ☐ Die Lösung wurde als Einzelleistung selbst erarbeitet.
- ☐ Kein über kleine Hilfsfunktionen oder Dokumentation hinausgehender Quellcode wurde KI-generiert.
- ☐ KI-Chatverläufe wurden als geteilter Link oder PDF-Dokument gesichert und mit abgegeben.

Datenmodell

- ☐ Der Anwendungsfall und die Problemstellung werden jeweils nachvollziehbar beschrieben.
- ☐ Es werden mindestens fünf Anforderungen als Fließtext oder als User Stories beschrieben.
- ☐ Das Datenmodell wird in einem UML-Diagramm vollständig beschrieben.
- ☐ Das Datenmodell umfasst mindestens zwei Microservices mit jeweils eigenen Datenbankentitäten.
- ☐ Die Entitäten werden mit Namen, Schlüsselfeldern, Nichtschlüsselfeldern und Fremdschlüssel beschrieben.
- ☐ Das Datenmodell zeigt die Beziehungen zwischen den Datenbankentitäten.

REST-Webservices

- ☐ Jeder Microservice besitzt eine REST-Schnittstelle für öffentliche Zugriffe auf seine Daten und Funktionen.
- ☐ Jeder Microservice besitzt eine vollständige und korrekte OpenAPI-Beschreibung seiner REST-Schnittstelle.
- ☐ Die REST-Schnittstellen erlauben nur Zugriffe auf von den anderen Microservices benötigte Daten.
- ☐ Jede Entität, sofern Sie in den REST-Webservices enthalten ist, besitzt eine eindeutige URL für ihre Einträge.
- ☐ Die HTTP-Methoden `GET`, `PUT`, `POST` und `DELETE` werden, wie in der Vorlesung besprochen, verwendet.
- ☐ Jeder REST-Aufruf wird im Erfolgsfall mit Statuscode 200 oder 201 beantwortet. Im Fehlerfall mit 4xx.

Funktionale Anforderungen

- ☐ Jeder Microservice besitzt eine eigene sqllite-Datenbank zur Ablage seiner Daten.
- ☐ Die Datenbankzugriffe sind so programmiert, dass SQL-Injections vermieden werden.
- ☐ Beim Ändern von Daten wird die referentielle Integrität durch die Datenbank gewährleistet.
- ☐ Jeder Microservice liest beim Start eine **`.env`**-Datei mit den Adressen der benötigten Fremdservices ein.
- ☐ Die Microservices greifen nur lesend auf die REST-Schnittstelle der anderen Microservices zu.
- ☐ Jeder Microservice sendet bei Änderungen an seinen Daten ein Event an den MQTT-Broker.
- ☐ Jeder Microservice verarbeitet die für ihn relevanten MQTT-Events, um seine eigenen Daten zu aktualisieren.
- ☐ Die Zugriffe auf andere Microservices sind fehlertolerant und führen bei Nichtverfügbarkeit nicht zum Absturz.

Clean Code

- ☐ Es gibt keine Variablen/Konstanten, die nicht verwendet oder nur befüllt, aber nicht gelesen werden.
- ☐ Es gibt keine auskommentierten Codeleichen oder nicht verwendete Klassen, Funktionen und Methoden.
- ☐ Doppelter Code wird so gut es geht vermieden. Die SOLID-Kriterien werden eingehalten.
- ☐ Anstelle des veralteten Schlüsselworts **`var`** werden **`let`** und **`const`** verwendet.
- ☐ Promises werden mit **`async`**/**`await`** anstatt mit Callback-Funktionen behandelt.
- ☐ Jede Klasse, Funktion und Methode besitzt zumindest einen kurzen, erklärenden JSDoc-Kommentar.
- ☐ Methoden und Funktionen besitzen nicht mehr als ca. 50 ausführbare Zeilen besitzen.
- ☐ Die Codezeilen sind nicht länger als ca. 120 Zeichen.
- ☐ Die gängigen Namenskonventionen für JavaScript-Code werden beachtet (dieselben wie in Java).
- ☐ Erwartbare Fehler werden mit **`try`**/**`catch`** abgefangen und sinnvoll behandelt.
- ☐ **`catch`**-Blöcke sind deshalb nicht leer, sondern loggen mindestens die Exception auf der Konsole.

Die in den Vorlesungen gezeigten Beispiele berücksichtigen all diese Best Practices. ☺

Docker Compose

- ☐ Jeder Microservice besitzt ein **`Dockerfile`**, um daraus ein Docker Image zu bauen.
- ☐ Das Projekt besitzt ein Docker Compose File, mit dem alle Images gebaut und ausgeführt werden können.
- ☐ Das Docker Compose File startet auch alle benötigten Zusatzservices (mit Ausnahme des MQTT-Servers).

Portfolioabgabe in Moodle

- ☐ Dem Quellcode liegt eine **`README`**-Datei bei, die erklärt, wie die Microservices gestartet werden können.
- ☐ Das Verzeichnis **`node_modules`** wurde vor der Abgabe gelöscht.
- ☐ Der Quellcode wurde als ZIP-Datei gepackt.
- ☐ Textdokumente, Office-Dateien und Zeichnungen wurden als PDF-Datei gesichert.
- ☐ Alle Dateien wurden rechtzeitig in Moodle hochgeladen.
- ☐ Die Lernkontrollen in Moodle wurden rechtzeitig bearbeitet.